

Latency Optimization of Cascaded Voice Dialogue Systems with a Pipeline-Parallel Streaming Architecture

1st Haihua Mo

School of Computer, Electronics
and Information, Guangxi University
Nanning, China
hua0424@163.com

2nd Zhengyou Liang

School of Computer, Electronics
and Information, Guangxi University
Nanning, China
zhyliang@gxu.edu.cn

Abstract—Cascaded ASR–LLM–TTS dialogue systems preserve modularity but leave speech recognition and prompt prefilling on the post-utterance critical path. We present a pipeline-parallel architecture that overlaps chunked Whisper recognition with incremental LLM prefilling. Silero VAD and a timestamp-aligned sliding window produce an append-only downstream transcript stream, while a reusable key–value cache moves most prompt processing into the user’s speaking time. We evaluate 1,132 synthesized MultiWOZ/CrossWOZ utterances, concatenated human-read speech from LibriSpeech and AISHELL-1, a matched LocalAgreement-2-style baseline, and a unified time-to-first-audio (TTFA) path. On the same 498-sample synthetic subset, the overall TTFT reduction is 74.3% on the original platform and 70.4% on a second dual-RTX 3090 platform. For Extra Long human-read speech, reductions are 67.5% on LibriSpeech and 65.7% on AISHELL-1. Streaming ASR accounts for most of the measured gain; incremental prefilling has a smaller independent effect under Qwen2-7B. Unified-timeline measurements reduce median TTFA from 22.27 s to 3.11 s, although TTS remains a variable bottleneck. Internal ASR hypotheses can still drift after commitment, but no downstream rollback is issued; an exploratory 50-sample semantic evaluation reports a mean response-embedding cosine of 0.883.

Index Terms—intelligent systems, human–computer interaction, voice dialogue system, streaming architecture, pipeline parallelism, incremental prefilling, end-to-end latency

I. INTRODUCTION

Spoken interfaces are used in service robots, intelligent transportation, and smart assistants. This work considers a conventional modular cascade composed of automatic speech recognition (ASR), a large language model (LLM), and text-to-speech (TTS). In the serial configuration studied here, downstream understanding and generation begin only after endpoint detection and full-utterance recognition, leaving recognition and prompt processing on the post-speech critical path. The resulting waiting time motivates the pipeline-parallel schedule evaluated in this paper.

Large-model voice interfaces improve the range of dialogue tasks that can be addressed but also introduce substantial inference cost. OpenAI reports that GPT-4o can respond

to audio input in as little as 232 ms, with an average of 320 ms [1]. These vendor-reported latency figures motivate treating response latency, alongside recognition and response quality, as an explicit evaluation dimension.

End-to-end multimodal models and modular cascades present different engineering trade-offs. The system studied here decomposes processing into independently configured VAD, ASR, LLM, and TTS modules connected in sequence, allowing the scheduling contribution to be evaluated without changing the underlying Whisper or Qwen2 weights. In the serial baseline, total system latency depends on both component processing times and the order in which data become available. For analysis, we express this serial latency budget as a simple sum:

$$T_{\text{total}} = T_{\text{VAD}} + T_{\text{ASR}} + T_{\text{Prefill}} + T_{\text{Decode}} + T_{\text{TTS}} + T_{\text{Net}} \quad (1)$$

Here T_{VAD} denotes voice-activity and turn-boundary processing, T_{ASR} speech recognition, T_{Prefill} prompt prefilling, T_{Decode} autoregressive decoding, T_{TTS} speech synthesis, and T_{Net} communication and serialization. Component overlap means that (1) is a conceptual serial budget rather than an identity for the proposed pipeline. We distinguish a conceptual physical-speech-end TTFT,

$$\text{TTFT} = t_{\text{first token}} - t_{\text{speech end}}, \quad (2)$$

from the metric available in the prerecorded legacy archives,

$$\text{TTFT}_{\text{post-feed}} = t_{\text{first token}} - t_{\text{feed end}}. \quad (3)$$

The latter is called *post-feed residual TTFT* throughout the legacy results. It includes pipeline closure, remaining ASR and prefill work, and first-token selection, but excludes TTS and should not be interpreted as a separately detected physical endpoint. For the unified run, server-side time to first playable PCM is

$$\text{TTFA}_{\text{server}} = t_{\text{first playable PCM}} - t_{\text{speech end}}. \quad (4)$$

Here “playable” means at least 1324 bytes, approximately 30 ms of 22.05-kHz mono 16-bit PCM. This timestamp additionally includes response-text readiness, the localhost TTS request, and initial synthesis, but excludes client transport, jitter buffering, audio-device scheduling, and acoustic playback. The prototype uses in-process queues for ASR/LLM communication and a localhost TTS service, so it does not measure a distributed network path or user-perceived audibility.

In conventional serial cascades, several seconds may remain between feed completion and server-side response readiness. For long-form input, full-audio ASR and one-shot LLM pre-filling both remain on that residual path. Reducing this server-side residual work is relevant to intelligent voice interfaces, although the present study does not measure user-perceived responsiveness or usability. The question addressed here is whether a modular cascade can move recognition and pre-fill work into the audio-input interval without changing the underlying ASR and LLM weights.

II. RELATED WORK

A. Streaming Progress in Automatic Speech Recognition

The processing speed of the ASR system determines how quickly user speech becomes text the LLM can consume. Early streaming ASR architectures prominently included the RNN-Transducer (RNN-T) [2]. Its recurrent structure supports incremental audio processing, although sequential dependencies limit parallelism over time. Transformer self-attention instead permits broad contextual interaction [3]; without modifications that restrict the available context or attention pattern, however, the vanilla formulation does not provide bounded-latency streaming inference.

Gulati et al. proposed Conformer, which combines convolutional local feature extraction with Transformer-based global modeling [4]. Streaming adaptations commonly constrain recognition to chunks or stabilize outputs across repeated partial hypotheses. For Whisper, the LocalAgreement policy provides one such output-stabilization approach [5], [6], while Simul-Whisper uses cross-attention guidance and truncation detection for incomplete chunk-boundary words [7]. Whisper itself is an encoder-decoder model developed primarily for offline transcription [8], so applying it incrementally requires explicit context and commitment policies. Our fixed-context timestamp policy is an engineering operating point rather than a claim to replace these agreement- or attention-guided methods.

B. LLM Inference Acceleration

Once ASR transcription completes, LLM inference can become another major latency bottleneck because autoregressive generation repeatedly accesses an expanding context and its cached state [9]. KV caching stores historical Key/Value states so that they need not be recomputed at every decoding step. vLLM applies PagedAttention to manage this cache for serving workloads [10], while Prompt Cache precomputes reusable prompt-module states and SGLang’s RadixAttention organizes shared request prefixes for automatic KV reuse [11],

[12]. These methods target reusable prompt structure or cross-request prefixes; our prototype instead appends newly committed speech fragments within one request and studies when that work is scheduled. For prompt processing, FlashAttention restructures exact attention around tiled memory access and reports improved throughput in evaluated settings [13]. StreamingLLM retains initial attention-sink tokens together with a recent-token window, enabling bounded-cache streaming over long sequences [14]; that bounded-context mechanism is not used here.

C. Full-Pipeline Optimization of Voice Dialogue Systems

Beyond optimizing ASR or the LLM in isolation, coordinated scheduling across the whole pipeline also matters for overall latency. In speech processing tasks such as speech translation, cascaded designs reuse separate components but may incur intermediate-representation loss, error propagation, and accumulated inter-module latency [15]. Recent speech-native dialogue systems instead model or generate speech within an integrated architecture [16]. Representative systems include Mini-Omni [17], Moshi [18], and LLaMA-Omni [19].

D. Contributions of This Work

Much prior work focuses on ASR-side streaming or on speech-native multimodal generation. Complementary to those directions, this paper proposes a fine-grained pipeline-parallel design (hereafter System B) that schedules an existing non-streaming ASR model and text LLM incrementally without changing their weights. Our main contributions are:

- **Streaming scheduling and state interface.** A VAD-driven fixed-context Whisper window combines prefix-suffix acoustic context and timestamp-aligned commitment with an append-only downstream text contract. Incremental LLM prefilling reuses historical KV state as a scheduling component; the evaluation does not assume that it is the dominant source of latency reduction.
- **Distributional latency evaluation.** In addition to the synthesized MultiWOZ/CrossWOZ stress benchmark, we evaluate concatenated LibriSpeech/AISHELL-1 human-read speech, noise and speed perturbations, repeated measurements, and a project-internal configured LocalAgreement-2-style comparator [5], [6]. We report dialogue-cluster inference on the locked 498-turn valid analysis set.
- **Policy-level first-audio and state analysis.** We measure server-side first-playable-PCM latency under explicitly unequal response policies, together with internal ASR drift, tokenizer seams, and exploratory response-level consistency. These measurements distinguish the append-only downstream contract from unsupported claims of internal hypothesis, KV-state, output-quality, or usability equivalence.

III. PRINCIPLES AND METHODS

A. Automatic Speech Recognition

1) *Whisper Architecture and Its Offline Nature:* We adopt OpenAI’s Whisper as the speech-to-text module. Whisper is

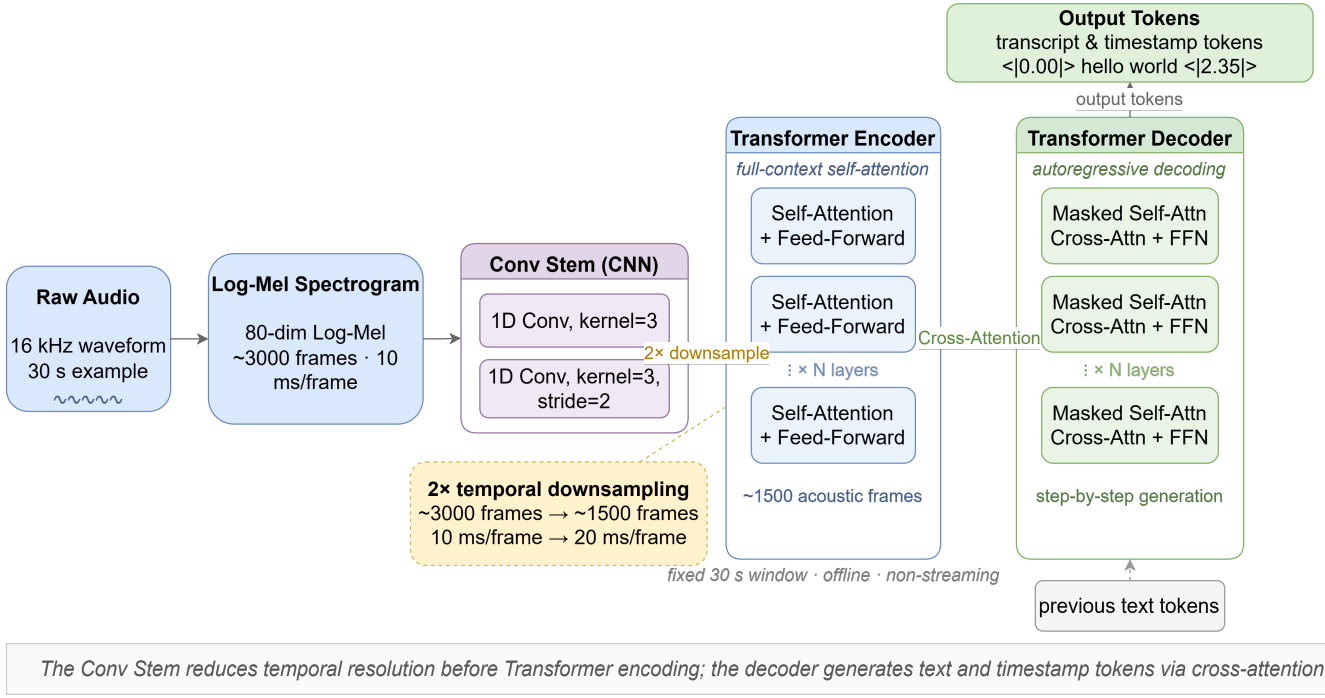


Fig. 1. Overall architecture of the Whisper model and schematic of feature downsampling

an end-to-end Transformer encoder–decoder model trained on large-scale weakly supervised speech–text data. Its primary report describes strong zero-shot performance across multiple languages and evaluated distribution-shift and noise benchmarks [8].

As shown in Fig. 1, the Whisper large-v3-turbo configuration uses 128-dimensional log-Mel features; this value is checkpoint-specific, whereas the original Whisper models used 80 Mel bins [20]. A convolutional front end downsamples these features before the Transformer encoder, and the decoder attends to the encoded representation while emitting text and timestamp tokens. These timestamps support the alignment policy used by our append-only interface. The archived run metadata identifies the model as *turbo* but does not preserve an immutable repository revision, so the cited public configuration documents the model family rather than the exact artifact downloaded during the experiments.

2) *Streaming Challenges and Our Approach*: Whisper’s encoder can use both past and future frames within an input window. When it is repeatedly applied to overlapping partial audio, words near chunk boundaries may be omitted, repeated, or revised, and the trailing hypothesis can change as more audio arrives [5], [6]. A downstream LLM therefore requires an explicit policy that decides which partial text is stable enough to commit.

We address both problems with segmented input, overlapped context, and controlled commitment. The front end uses VAD to split PCM into speech segments; the ASR side retains acoustic context around the target; and the output side uses word-level timestamps to commit only the current output

region. Once delivered, a fragment is not withdrawn, even if later internal recognition changes that region.

B. LLM Inference

1) *Transformer Attention*: Large language models typically adopt a decoder-only Transformer architecture. Within the LLM alone, first-token computation consists mainly of prompt prefill and the first decoding step. The system-level TTFT in (2) additionally includes pipeline closure and the ASR tail. The core of the Transformer is scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (5)$$

Here Q (Query), K (Key), and V (Value) are obtained from the input vectors through linear projection matrices W_Q , W_K , and W_V , respectively. The Key dimension d_k rescales the dot products so that $QK^\top$ does not suffer variance inflation as dimensionality grows, which would saturate the softmax and shrink gradients. This mechanism lets the model aggregate information from any position of the sequence within a single time step, capturing long-range dependencies.

2) *KV Cache: Mathematical Principle and Complexity*: Text generation in an LLM is autoregressive: to generate the N -th token, the model uses the preceding context. Under dense self-attention, a full uncached forward pass over a sequence of length N has an $O(N^2)$ attention cost [3]. If such a full pass

were naively repeated at every generation step, the illustrative cumulative cost would be

$$T_{\text{total}}^{\text{naive}} = \sum_{t=1}^N O(t^2) = O\left(\frac{N(N+1)(2N+1)}{6}\right) \approx O(N^3) \quad (6)$$

Modern inference frameworks avoid literally repeating those full forward passes by caching historical Key/Value states [10]. The preceding cubic expression is therefore an intentionally naive reference model, not the complexity of the implementation evaluated here. With cached causal decoding, the model computes new Key/Value states for the current token and appends them to the stored history:

$$K_{\text{cache}}^{(t)} = \text{Concat}\left(K_{\text{cache}}^{(t-1)}, k_t\right) \quad (7)$$

$$V_{\text{cache}}^{(t)} = \text{Concat}\left(V_{\text{cache}}^{(t-1)}, v_t\right) \quad (8)$$

Attention then involves only the interaction of the current query q_t with the cached history K_{cache} and V_{cache} : compute $q_t K_{\text{cache}}^\top$ and take the weighted sum over V_{cache} . The KV cache thus lowers the per-step decoding complexity from $O(t^2)$ to $O(t)$ —still linear in context length—so the cumulative cost of generating N tokens is about $\sum_{t=1}^N O(t) = O(N^2)$, at the price of cache memory that grows linearly with sequence length.

The KV cache plays two distinct roles here. During autoregressive decoding, both systems reuse historical Key/Value states. System B additionally applies cache reuse to prompt arrival: whenever a committed transcript fragment arrives, only its token sequence is forwarded and appended to the cached prompt state. This does not eliminate attention computation or change the asymptotic order of processing the complete input. It relocates prompt work into the speaking interval and thereby reduces the computation remaining after speech end. Separately tokenizing fragments is not guaranteed to produce the same token sequence as tokenizing their concatenation once; therefore, the implementation claims append-only downstream state, not token-sequence, KV-state, or output-distribution identity with one-shot preprocessing. Fig. 2 contrasts one-shot prompt prefill in System A with incremental prompt prefill in System B.

C. Evaluation Metrics

1) *Time to First Token and Time to First Audio*: Legacy prerecorded results follow (3); the unified run follows (4), with all timestamps on one `perf_counter_ns` clock. Its closed server-side timeline is physical speech end $\rightarrow$ feed end $\rightarrow$ pipeline input close $\rightarrow$ first token $\rightarrow$ policy-defined response text ready $\rightarrow$ TTS request $\rightarrow$ first playable PCM. We define

$$t_{\text{feed_to_close_wait}} = t_{\text{input_close}} - t_{\text{feed_end}}. \quad (9)$$

This interval includes scheduling before and after explicit flush and must not be interpreted as flush computation alone.

2) *Recognition and Statistical Reporting*: Word error rate (WER), the standard metric for ASR accuracy, is computed from the Levenshtein edit distance:

$$\text{WER} = \frac{S + D + I}{N_{\text{ref}}} \quad (10)$$

where S , D , and I are substitution, deletion, and insertion counts and N_{ref} is the reference length. We primarily report WER for English and CER for Chinese. Table labels distinguish mean-utterance error (the arithmetic mean of sample-level rates) from corpus error $\sum(S+D+I)/\sum N_{\text{ref}}$. Latency tables report the sample mean, sample standard deviation (ddof=1), and tail percentiles. Because multiple accumulated turns share a source dialogue, primary synthetic-latency uncertainty uses a one-stage paired dialogue-cluster bootstrap: the (dataset, dialogue) clusters are sampled with replacement, all paired turns in each selected cluster are retained, and the ratio-of-pooled-means reduction $1 - \bar{T}_B/\bar{T}_A$ is recomputed for 10,000 resamples with seed 20260821. Dialogue-level Wilcoxon tests operate on per-dialogue mean paired differences, and Holm correction is applied within each declared duration or ablation family. This keeps the original turn-weighted point estimate while accounting for shared dialogue ancestry in uncertainty. Fig. 6 shows observed P5–P95 bands, not confidence intervals.

IV. SYSTEM DESIGN

A. Overall Architecture

1) *Objective and Logical Topology*: The core design objective is to shift from the traditional “receive everything, then process everything” paradigm to an “incremental receive, streaming process” paradigm, reducing the work that remains after waveform feed completion. The logical topology is shown in Fig. 3, which depicts the streaming parallel data flow built on multi-threaded producer–consumer queues. Audio enters the segmentation module continuously as fixed-length PCM chunks; Silero VAD performs activity detection on an accumulation buffer and emits speech segments [21]. The ASR module concatenates several speech segments, invokes Whisper for transcription, and emits append-only text fragments according to the prefix/suffix context policy. Text fragments are passed through a thread-safe queue to the LLM module, which keeps updating the KV cache and starts generation when the termination marker arrives, thereby moving as much LLM prefill computation as possible forward to overlap with audio input.

The overall architecture is a segment–transcribe–prefill–generate pipeline: as soon as an upstream module produces a new audio or text segment, the downstream module can start computing without waiting for the full input. The system comprises three subsystems: a Streaming Audio Segmenter for voice activity detection and segmentation, a Context-Aware ASR Engine for context concatenation and output stability, and an Incremental LLM Inference Service for incremental prefilling and final generation. The KV cache in the LLM part is a general optimization for Transformer decoder inference;

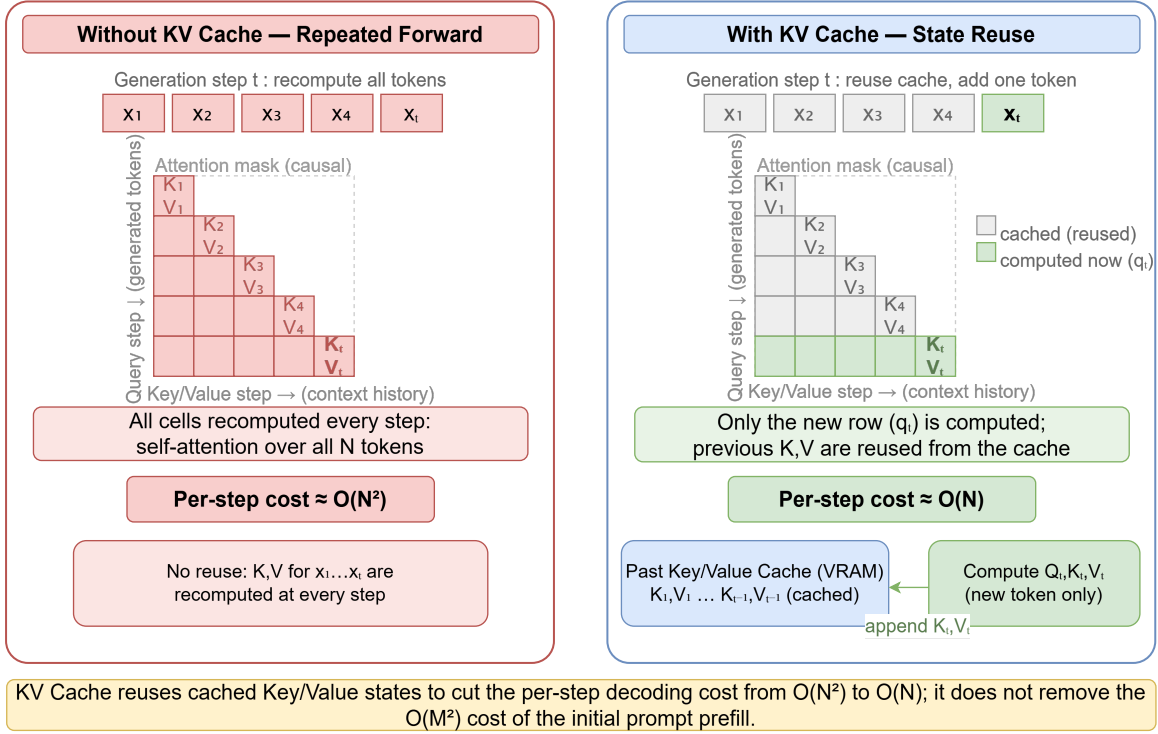


Fig. 2. Prompt-prefill scheduling. System A prefills the complete prompt after speech end; System B prefills committed fragments incrementally. Both systems use standard KV-cache reuse during autoregressive decoding.

we directly reuse the `past_key_values` mechanism built into the inference framework for state reuse.

2) *Key Engineering Details*: To keep the streaming pipeline real-time and controllable in a single-machine prototype, the implementation splits the chain into a multi-threaded pipeline decoupled by thread-safe queues, with state objects carrying cross-segment context.

For communication, modules do not call each other synchronously; they communicate asynchronously through a producer-consumer model. In the implementation, the audio-chunk queue, audio-segment queue, and text-segment queue are all instances of `queue.Queue`; consumers loop on `get` with a timeout, and a designated end-event marks the end of production, avoiding single-point blocking that could deadlock the pipeline. In the ASR stage, the system must simultaneously accept segmented audio from upstream and stream transcribed text downstream, so it is split into two sub-threads, a collector and a transcriber: the former continuously receives upstream speech segments and writes them to a waiting queue; the latter, when the trigger condition is met, concatenates them in batch, calls Whisper for transcription, and pushes the current commit region to the downstream LLM, decoupling audio collection from model inference.

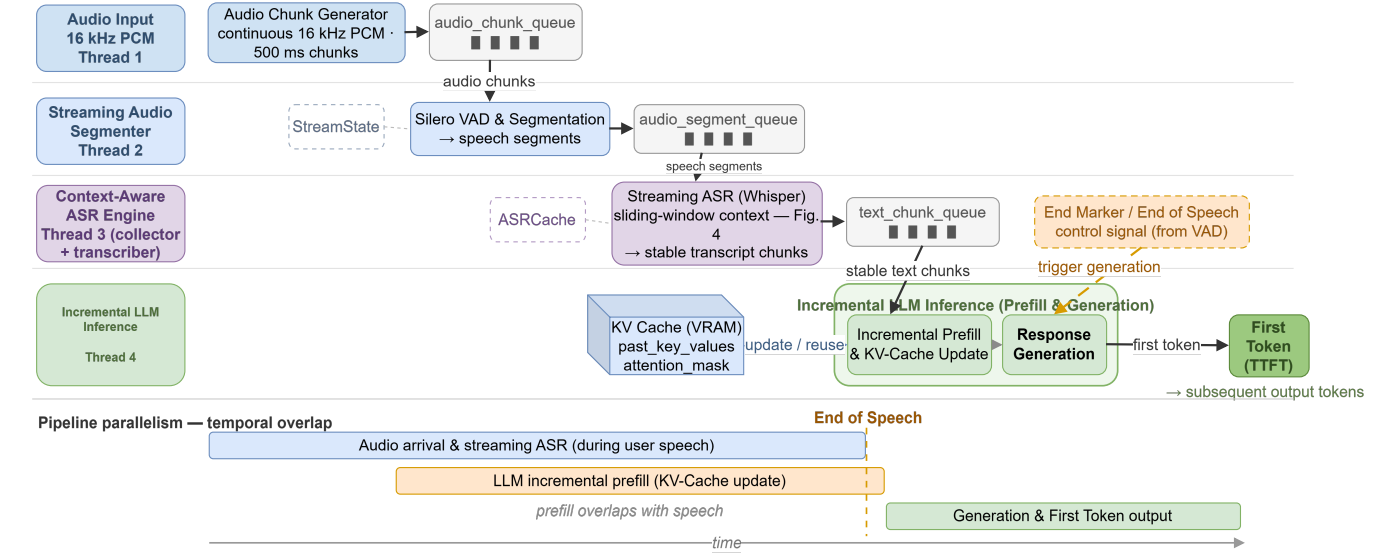
State management uses a hybrid stateful/stateless design. The segmentation module maintains the accumulated audio buffer, segment boundaries, and indices in `StreamState`. The ASR side manages pending and transcription-window queues in `ASRCache`. The LLM per-

sists `past_key_values`, `attention_mask`, and last-step prefill logits in `KVCache`. Together these objects maintain cross-segment state and an append-only downstream interface; they do not guarantee that later internal ASR hypotheses remain unchanged.

B. Streaming ASR Context Management

1) *Strategy*: Streaming ASR exposes a configuration-dependent trade-off between the amount of acoustic context available to the recognizer and the delay before a fragment can be processed. All reported runs use the historical implementation’s latched startup-duration policy. Its `total_duration` counter accumulates the durations of speech segments admitted to the ASR cache and is not decremented when processed segments are removed. The 2.0-s condition is therefore a one-time cumulative-seen-duration startup gate: after its first crossing, the gate remains open and later invocations depend mainly on segment-count eligibility or final-input processing. We report measurements of this historical policy and make no performance claim for an unmeasured corrected current-queue-duration policy.

2) *Dynamic VAD Segmentation*: The system uses Silero VAD to segment the continuous PCM stream online [21]. Define the input audio stream as a time series $\{x_1, x_2, \dots, x_t\}$. The segmenter maintains an accumulation buffer A_{accum} and, on receiving each new audio chunk c_t , performs $A_{\text{accum}} \leftarrow A_{\text{accum}} \oplus c_t$, where $\oplus$ denotes concatenation. Once the accumulated audio exceeds the detection window, Silero VAD



Producer-consumer queues let ASR transcription and LLM prefill overlap with the user's speech; after End-of-Speech only the last chunk and thread synchronization remain on the critical path.

Fig. 3. Logical topology of the streaming parallel architecture

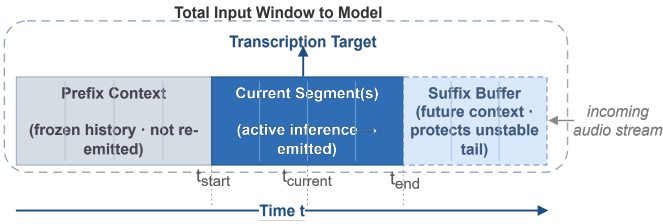


Fig. 4. Schematic of the context-aware ASR sliding-window mechanism

is invoked; a closed speech segment is then emitted to the downstream ASR. The legacy post-feed TTFT, ablation, context, human-read, and comparator runs use 500-ms chunks, a Silero threshold of 0.5, a 500-ms minimum speech duration, a 300-ms minimum silence duration, and the 2.0-s latched cumulative-seen-duration startup gate described above. The R7 unified run instead records a Silero threshold of 0.5, a 250-ms minimum speech duration, a 100-ms minimum silence duration, and 30-ms speech padding, while retaining the 500-ms chunk and the same latched startup gate. These parameters are fixed within each paired comparison. Because no VAD-parameter ablation was performed, we do not claim that the selected values optimize responsiveness or segmentation stability.

3) *Sliding-Window Context Awareness:* To reduce boundary effects in streaming recognition, the ASR engine uses a fixed-context sliding window with prefix context. Fig. 4 details the acoustic window at time step t ; it does not depict the separate latched startup gate. The window contains frozen prefix audio, the target segment, and an optional suffix buffer. This overlap is intended to reduce boundary instability; it does not guarantee that later internal re-recognition will reproduce

the same text.

First, the window is updated. Let Q_t be the audio-segment queue at time t . Newly arrived segments first enter a waiting queue and are then merged in batch into the main queue by the transcription thread, yielding Q_t . Before its first crossing, the historical startup gate requires cumulative admitted speech duration to reach $\tau_{\text{start}} = 2.0$ s. Because the counter is not reduced when old segments leave Q_t , this duration condition stays true thereafter. Non-final invocations then require the segment-count eligibility condition

$$|Q_t| \geq N_{\text{prefix}} + 1 + N_{\text{suffix}}, \quad (11)$$

where N_{prefix} and N_{suffix} are the configured numbers of preceding and following segments. Final-input processing bypasses these gates. When eligible, the ASR model concatenates the current queue and performs one transcription pass to produce the raw transcript. Thus (11) is the current-queue component of the measured policy, whereas the 2.0-s value is a latched startup condition rather than a recurring current-window-duration threshold.

Second, timestamp-aligned text extraction maps Whisper words to segment spans by word end time and emits only segments inside the current commit region. The first and final rounds receive explicit boundary handling so no trailing text is intentionally withheld. After emission, the window retains configured prefix and uncommitted suffix segments. A larger suffix can reduce some boundary errors but also delays commitment and adds decoding work; Table V shows that its effect is metric- and language-dependent.

4) *Commit Contract and Tokenizer Seams:* The ASR maintains two state layers. Internal hypotheses may be overwritten when retained audio is recognized again. In contrast, a committed downstream fragment is delivered once and never

retracted. There is no KV-cache rollback path: corrections observed internally after commitment are logged but are not propagated to the LLM. Thus “append-only” describes the downstream interface, not immutable internal recognition. A tokenizer seam occurs when fragment-wise tokenization differs from one-shot tokenization,

$$\mathcal{T}(f_1) \parallel \mathcal{T}(f_2) \neq \mathcal{T}(f_1 \parallel f_2). \quad (12)$$

The cache stores the actual fragment-wise token sequence. Measurements of internal drift and seam frequency are reported in Section V-F.

C. Incremental KV-Cache Prefilling for the LLM

1) *Strategy*: In System A, the LLM obtains the complete prompt only after ASR finishes transcribing the whole utterance, and performs prefill plus first-token generation after waveform feed completion in the legacy experiment. Post-feed residual TTFT therefore rises markedly as transcript length grows. Applying that serial paradigm unchanged to a setting where computation could proceed during audio input leaves available overlap unused.

System B reduces residual computation by retaining the `past_key_values` produced by each committed fragment. A new fragment triggers a forward pass only for its own tokens, after which the updated cache replaces the previous state. When input closes, any remaining fragment and the generation prompt are processed before decoding begins. The amount of work left at closure depends on the final segment and queue state; the design can move prefill into the audio-input interval, but the isolated measured contrast is small and non-monotonic in the present ablation. Because the cache is append-only, revising an already delivered fragment would require rollback or recomputation, which this prototype does not implement.

2) *Core Algorithm*: The core logic of incremental inference: on the first call, the system performs a one-shot prefill without any cache, establishing the initial KV cache; thereafter, each new ASR text fragment from upstream triggers a call to `cache_prompt()`, which updates incrementally on top of the existing `past_key_values`. When the termination marker is received, the system appends the generation prompt (e.g., “assistant:”, telling the model that the next output token begins the assistant’s reply) to the end of the prompt and reuses the last-step logits from prefilling to start decoding directly. To avoid positional-encoding misalignment of incremental tokens, the implementation constructs `position_ids` explicitly and encodes new fragments without adding another generation prompt, keeping cache concatenation controlled. Algorithm 1 gives the incremental update procedure corresponding to this implementation. Fig. 5 visualizes the state changes in GPU memory: the blue region on the left is the cached key-value pairs of the historical context, and the green region on the right is the key-value pairs produced by the new fragment. On each fragment’s arrival, incremental prefilling reuses the historical `past_key_values` and runs the forward pass only over new tokens before updating the

Algorithm 1 Incremental KV cache prefill (StreamLLMInference)

Require: current KV cache $C_{\text{prev}} = (P_{\text{prev}}, \text{Mask}_{\text{prev}}, L_{\text{prev}})$; new text fragment T_{new} ; end flag IsEnd ; generation prompt G ; tokenizer $\mathcal{T}$; LLM $\mathcal{M}$

Ensure: updated cache $C_{\text{new}} = (P_{\text{new}}, \text{Mask}_{\text{new}}, L_{\text{new}})$

- 1: **Step 1: fragment preparation and tokenization**
- 2: **if** $\text{IsEnd} = \text{True}$ **then**
- 3: $T_{\text{new}} \leftarrow \text{Concat}(T_{\text{new}}, G)$ $\triangleright$ append generation prompt
- 4: **end if**
- 5: **if** T_{new} is empty **then**
- 6: **raise exception** $\triangleright$ invalid empty fragment
- 7: **end if**
- 8: $\text{Ids}_{\text{new}} \leftarrow \mathcal{T}(T_{\text{new}}, \text{add_special_tokens} = \text{False})$
- 9: **if** Ids_{new} is empty **then**
- 10: **raise exception** $\triangleright$ no valid token in fragment
- 11: **end if**
- 12: **Step 2: attention mask and position IDs**
- 13: $\text{Mask}_{\text{new}} \leftarrow \text{Concat}(\text{Mask}_{\text{prev}}, \mathbf{1}_{|\text{Ids}_{\text{new}}|})$
- 14: $\text{Pos} \leftarrow [|\text{Mask}_{\text{prev}}|, \dots, |\text{Mask}_{\text{new}}| - 1]$ $\triangleright$ explicit position_ids
- 15: **Step 3: forward pass (incremental update)**
- 16: $\text{Outputs} \leftarrow \mathcal{M}(\text{Ids}_{\text{new}}, \text{Mask}_{\text{new}}, \text{Pos}, P_{\text{prev}}, \text{use_cache} = \text{True})$
 $\triangleright$ forward only on new tokens, reusing cached P_{prev}
- 17: **Step 4: cache and logits update**
- 18: $P_{\text{new}} \leftarrow \text{Outputs.past_key_values}$
- 19: $L_{\text{new}} \leftarrow \text{Outputs.logits}[:, -1, :]$ $\triangleright$
if $\text{IsEnd} = \text{True}$, the first token is decoded from L_{new} without an extra forward pass
- 20: $C_{\text{new}} \leftarrow (P_{\text{new}}, \text{Mask}_{\text{new}}, L_{\text{new}})$
- 21: **return** C_{new}

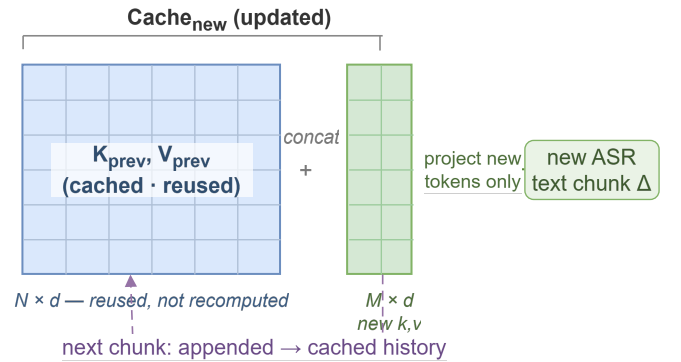


Fig. 5. Schematic of the LLM KV Cache incremental update mechanism

cache, avoiding repeated prefills over the full prompt in the incremental setting.

3) *Complexity Analysis*: To locate the source of the redundancy that incremental prefilling removes, consider one update with historical context length N and new-fragment length M .

If every incremental arrival triggered a fresh prefill over the whole sequence $x_{1:N+M}$ (no reuse of historical state), the dominant self-attention cost would be about $O((N+M)^2)$. With the KV cache, the K, V of historical tokens are already stored; the system only computes queries for the new tokens and lets them interact with Key/Value of length $N+M$, at a dominant cost of $O(M(N+M))$. When $M \ll N$ this approximates $O(MN)$, i.e., it is approximately bilinear in the new-fragment and history lengths and linear in either one only when the other is treated as fixed. Note that if all incremental fragments accumulate to a final length L , the cumulative cost of incremental prefilling is

$$\sum_j O(M_j L_j) = O(L^2) \quad (13)$$

where M_j and L_j are the length of the j -th new text fragment and the accumulated length, and L is the full sequence length—the same order as offline one-shot prefilling. The intended benefit is not a lower complexity order but relocation of prefill computation into the audio-input interval, which can reduce work remaining at closure. The measured isolated effect is reported rather than inferred from this mechanism. The implementation also retains the last-step logits from prefill and samples the first generated token from them after the termination marker; this is standard causal-LM prompt processing and is not treated as an independently established advantage over System A.

D. Test Data Construction Pipeline

1) *Overview*: Systematically validating the streaming architecture under controlled conditions requires dialogue data of steadily increasing length, so we built an automated data-synthesis pipeline. It converts two standard multi-turn dialogue datasets, MultiWOZ (English) [22] and CrossWOZ (Chinese) [23], into progressively lengthening long-speech test data with precise duration annotations, and then generates audio via TTS.

2) *Cumulative Dialogue Generation*: MultiWOZ and CrossWOZ both contain multi-turn dialogues, but individual user turns are usually short and cannot directly cover long-speech input. To construct controlled long-input stress cases from semantically related material, we use a cumulative dialogue strategy; this construction is not intended to reproduce a naturally observed single-speaker turn. As illustrated in Table I, the algorithm walks through the dialogue history turn by turn, concatenating the user-side inputs and system replies of the current and preceding turns into input sequences of increasing length.

This strategy generates continuous speech samples from 3 seconds to over 60 seconds grounded in real semantic context, covering interaction forms from short commands to extended statements and remedying the lack of long-speech samples in conventional datasets.

3) *Data Processing Pipeline*: Data construction proceeds in three strictly ordered stages.

The first is purposive source-dialogue selection, history accumulation, and filtering. For each corpus, source dialogues are ranked by their total dialogue-text length and the longest selected dialogues are traversed turn by turn. Multiple accumulated-turn samples are then emitted from each selected dialogue, subject to maximum per-sample text lengths of 2,050 characters for English and 720 for Chinese and an upper speech-duration bound. Consequently, accumulated turns from one dialogue share substantial prefixes and are statistically clustered; the 1,132-turn primary set comes from 100 source dialogues (cluster size 6–18, median 11, mean 11.32). This is a controlled long-input stress benchmark rather than a sample from a natural conversational-turn distribution, and aggregate results depend on its duration mix.

The second stage is concurrent TTS synthesis. We use a CosyVoice-based HTTP synthesis endpoint to produce controlled synthetic waveforms from the accumulated text [24]. A batch client issues asynchronous requests to the endpoint; the resulting audio remains synthetic and is not treated as human speech. The archived artifacts do not preserve an immutable CosyVoice checkpoint or service revision, so [24] identifies the model family rather than a specific hosted Alibaba service.

The third is duration calibration and metadata synchronization. Because generative TTS speaking rate is nondeterministic, estimating duration from character counts is error-prone. The pipeline parses the generated WAV file headers to obtain physical durations in seconds (accurate to milliseconds), which are written back to the test-metadata JSON as ground truth for the X-axis (input duration) of subsequent experiments.

4) *Test-Set Groups*: For fine-grained analysis of latency across durations, the generated samples are divided into five predefined analysis groups by audio duration (Table II).

This grouping is used throughout the experimental analysis in Section V to compare how the architectures handle speech of different lengths.

5) *Concatenated Human-Read Speech and Augmentation*: To test acoustic conditions not represented by TTS, we construct long-form samples from LibriSpeech test-clean [25] and AISHELL-1 [26]. LibriSpeech utterances are concatenated in chapter order and AISHELL-1 utterances in speaker order. Uniformly sampled silences $U(0.2, 1.0)$ s are inserted between source utterances with seed 42. Each language contributes 75 clean samples (30 Long, 30 Very Long, and 15 Extra Long), adding multiple speakers and recording conditions; AISHELL-1 further broadens Mandarin speaker and accent coverage. For each language, six 30-sample variants add MUSAN [27] noise at SNR 20, 15, and 10 dB, apply speed factors 0.9 and 1.1, or add 15-dB babble. Speed-modified samples are regrouped using their new durations. A five-sample manual listening check passed intelligibility and alignment checks, while confirming that concatenation seams can be audible. We therefore describe these data as concatenated human-read speech rather than spontaneous dialogue.

TABLE I
CUMULATIVE TURNS FROM ONE SOURCE DIALOGUE; ROWS SHARE HISTORY AND BELONG TO ONE STATISTICAL CLUSTER (EXAMPLE TRANSLATED FROM CHINESE)

Turn	User	Agent	Accumulated text
1	Book a flight	Where to?	Book a flight
2	To Beijing, leaving tomorrow	Which cabin class?	Book a flight, Where to?, To Beijing, leaving tomorrow
3	Business class	OK, booking it now	Book a flight, Where to?, To Beijing, leaving tomorrow. Which cabin class? Business class

TABLE II
CONTROLLED AUDIO-DURATION ANALYSIS STRATA

Group	Duration (s)	Analytical interpretation	Experimental focus
Short	$T < 5$	Controlled short input	Fixed-overhead regime
Medium	$5 \leq T < 15$	Controlled medium input	Crossover behavior
Long	$15 \leq T < 30$	Controlled long input	Residual-latency scaling
Very Long	$30 \leq T < 60$	Controlled extended input	Tail-latency growth
Extra Long	$T \geq 60$	Extreme stress input	Stability and OOM boundary

V. EXPERIMENTS AND ANALYSIS

A. Experimental Protocol

System A is a serial non-streaming cascade: full-audio Whisper large-v3-turbo transcription is followed by one-shot Qwen/Qwen2-7B-Instruct [28] prompt prefill and generation. System B uses the same loaded weights but applies the historical latched-gate streaming ASR policy and incremental prefill mechanisms described above. The archived run metadata records the model identifiers as `turbo` and `Qwen/Qwen2-7B-Instruct`; immutable model-repository revisions were not preserved and are therefore reported as unavailable rather than inferred retrospectively. Tables III–V and Fig. 6 use the original 2025 dual-RTX 3090 platform. Repeated measurements and Tables VI–VIII use a second platform, a KVM virtual machine with an Intel Xeon Gold 6133 at 2.50 GHz and two RTX 3090 GPUs. Absolute latency is platform-dependent, so values from the two platforms are neither scaled nor mixed within a comparison. In both environments, Whisper runs on one GPU and Qwen2-7B-Instruct on the other, batch size is one, and three real-audio warm-up runs precede measurement. Legacy main runs use the archived fixed B-then-A order, while the three-arm ablation uses Baseline, Streaming-ASR-Only, then Full-Streaming order; this limits causal interpretation of small marginal contrasts. For R7 specifically, ASR and the localhost CosyVoice service share GPU0 and Qwen2 uses GPU1. “Exclusive” means that no unrelated GPU workload was present, not that ASR lacked TTS co-residency.

Tables III–V use a 50-token generation cap. The state, semantic, and unified TTFA experiments use 128 tokens. Effective sampling uses temperature 0.1 and top- p 0.9. Although the historical interface accepted `repetition_penalty=1.1`, post-experiment inspection found that the custom decoder did not apply it; all compared arms therefore used no effective repetition penalty. The original-platform mixed synthetic set

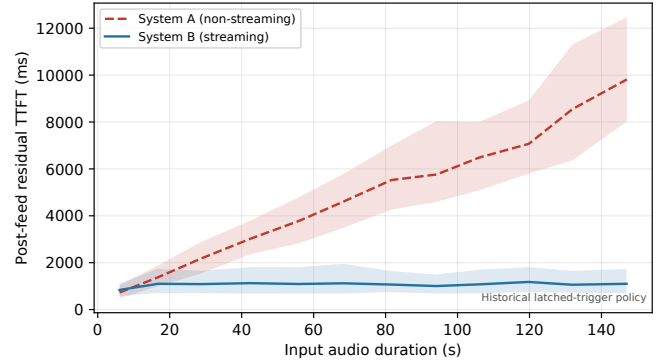


Fig. 6. Legacy post-feed residual TTFT for the historical latched-trigger implementation over 12 equal-frequency duration bins. Lines show bin means and shaded regions show observed P5–P95 ranges, not confidence intervals.

contains 1,132 samples: 630 from MultiWOZ and 502 from CrossWOZ. Paired exclusions are stated with each experiment.

B. Latency versus Speech Length

System B adds fixed overhead for Short and Medium samples, but its mean post-feed residual TTFT changes little over the three longer groups on the original platform. The Short, Medium, Long, Very Long, and Extra Long rows contain 35/35, 89/83, 121/92, 208/99, and 679/100 accumulated turns/source dialogues, respectively. For Long, Very Long, and Extra Long input, descriptive mean reductions are 34.6%, 65.6%, and 83.9%, respectively. Dialogue-cluster inference is reported on the locked 498-turn three-arm subset in Table IV; these broader Table III rows remain descriptive because their archived filter differs. The Extra Long absolute reduction is 5657.9 ms (5.66 s). Tail latency is not constant: System B P99 rises from 1978.67 to 2605.45 ms between Long and Extra Long. This $1.32\times$ growth is smaller than System A’s $4.96\times$ growth on the locked valid set. The empirical crossover

TABLE III
POST-FEED RESIDUAL TTFT BY DURATION FOR THE HISTORICAL LATCHED-TRIGGER IMPLEMENTATION (MS)

Group	n	System B			System A		
		Mean $\pm$ std	P95	P99	Mean $\pm$ std	P95	P99
Short	35	648.17 $\pm$ 70.76	744.76	797.17	533.42 $\pm$ 65.44	631.82	674.06
Medium	89	959.53 $\pm$ 184.10	1261.82	1397.39	923.42 $\pm$ 170.86	1196.04	1281.49
Long	121	1126.63 $\pm$ 323.72	1883.55	1978.67	1722.04 $\pm$ 342.63	2310.40	2474.07
Very Long	208	1099.16 $\pm$ 343.51	1768.83	2173.97	3191.97 $\pm$ 697.81	4467.14	4826.85
Extra Long	679	1087.70 $\pm$ 383.16	1717.10	2605.45	6745.57 $\pm$ 2108.79	11313.22	12274.38

depends on platform, fixed execution order, and speech distribution; the 15-s group boundary is not asserted as a universal routing threshold.

1) *Repeated Measurements*: On the second platform, 50 fixed Very Long accumulated turns from 40 source dialogues were run three times per mode under the same historical latched policy. System B post-feed residual TTFT round means were 1434.0, 1482.1, and 1454.9 ms, a range equal to 3.3% of the grand mean; System A means were 4063.3, 4244.2, and 4118.5 ms. With sample standard deviation, System B per-sample coefficient of variation (CV) had mean 5.19%, median 4.05%, P90 10.73%, and maximum 18.96%; the corresponding System A values were 5.23%, 4.65%, 9.92%, and 14.01%. Central behavior was repeatable, but both modes retained a non-negligible high-variance tail.

C. Ablation and Context Trade-offs

Table IV summarizes the historical three-arm ablation. The candidate pool contained 505 Long-or-longer turns. Run-log review identified seven executions affected by concurrent external programs rather than the evaluated systems; these contaminated runs were excluded before analysis. The locked valid set therefore contains 498 turns from 99 source dialogues (108 Long, 150 Very Long, and 240 Extra Long; cluster size 1–9, median 5). All reported ablation and cross-platform comparisons use this same 498-turn set.

Across the filtered 498 turns, the Baseline-minus-Streaming-ASR-Only contrast is 3332.13 ms with dialogue-cluster 95% CI [3064.38, 3572.61] ms, while the Streaming-ASR-Only-minus-Full contrast is only 15.50 ms [10.95, 19.66] ms. These are archived fixed-order arm contrasts, not causal component effects. The duration-stratified point estimates remain non-monotonic for incremental prefill: -23.01 , 2.73 , and 40.82 ms for Long, Very Long, and Extra Long; the Long result favors ASR-only and the Extra Long value is only 3.3% of ASR-only TTFT. Thus incremental prefill remains a scheduling component, but a stable large independent post-feed TTFT benefit is not established for Qwen2-7B under this historical policy.

Each context configuration contains 150 accumulated turns from 85 source dialogues (74 MultiWOZ and 76 CrossWOZ turns). Error cells are mean-utterance metrics; the ASR time column pools the streaming and non-streaming records ($n = 300$) and is not interpreted as streaming tail latency alone. Adding one suffix segment reduces CrossWOZ WER/CER from 0.0118/0.0292 to 0.0083/0.0203 while increasing pooled

ASR time. Removing all extra context is faster but raises CrossWOZ WER/CER to 0.0210/0.0500. MultiWOZ WER is not monotonic in context size. These data establish a measured trade-off on synthetic speech, not a universal language-specific configuration rule. System A’s near-zero error in this experiment is specific to TTS generated from its reference text.

D. Concatenated Human-Read Speech

On clean LibriSpeech, Extra Long mean post-feed residual TTFT is 4805.1 ± 420.2 ms for System A and 1559.0 ± 490.3 ms for System B, a 67.5% reduction. On clean AISHELL-1 the corresponding values are 5139.9 ± 381.7 and 1762.9 ± 275.4 ms, a 65.7% reduction. The clean human-read System A errors are no longer near zero: mean-utterance WER is 2.97% on LibriSpeech and CER is 10.77% on AISHELL-1; System B obtains 5.65% and 11.80%. No recognition-quality noninferiority margin or task-success criterion was prespecified, so these latency results do not establish quality preservation. AISHELL-1 references spell numbers in Chinese while Whisper often emits Arabic numerals; this affects 49/75 clean samples (mean CER 0.1476 when affected versus 0.0313 otherwise).

Across the ten SNR and speed conditions, TTFT reduction ranges from 22.8% to 31.6%, with Holm-adjusted paired tests at $p \leq 0.0129$. Babble is a boundary rather than a success case. For English, System B is 47.3% slower in the unpaired overall summary; paired inference on 29 common samples has a confidence interval reaching zero. Chinese babble yields only a 6.5% mean reduction and is not significant ($p = 0.271$). VAD over-triggering creates segment backlog, while Whisper produces empty streaming outputs for 12/30 English and 5/30 Chinese samples. English-babble System B has median 2020.4 ms but mean 3425.1 ms, and the slowest sample takes approximately 21 s after audio completion to drain the ASR queue.

E. Configured LA-2-Style Comparator

We implement a project-internal LocalAgreement-2-style comparator following the partial-hypothesis policy of Liu et al. [5] and its Whisper adaptation [6]. It uses the same Whisper weights, inference engine, audio segmentation, second platform, and 498-turn/99-dialogue filtered subset as System B, with absolute-time-axis commitment, punctuation-robust agreement, sentence-boundary-aware trimming, and a 15-s retained-buffer cap. Trigger policies are intentionally not identical: System B uses the historical latched startup gate,

TABLE IV
HISTORICAL-LATCHED-POLICY ABLATION ON THE ORIGINAL PLATFORM; LATENCY CELLS ARE MEAN $\pm$ STD [P95/P99] MS

Group	Baseline	Streaming ASR only	Full streaming	Base. — ASR	ASR — Full
Long	1690.81 $\pm$ 349.51 [2323.60/2431.03]	1064.67 $\pm$ 307.62 [1691.03/1873.56]	1087.69 $\pm$ 312.46 [1748.08/1859.16]	626.13	−23.01
Very Long	3307.96 $\pm$ 872.57 [4891.94/5468.95]	1155.09 $\pm$ 365.67 [1859.20/2259.80]	1152.36 $\pm$ 363.23 [1828.80/2228.55]	2152.87	2.73
Extra Long	6515.67 $\pm$ 1762.26 [9208.54/9619.23]	1228.82 $\pm$ 411.10 [1981.51/2858.89]	1188.00 $\pm$ 411.07 [1946.51/2812.95]	5286.86	40.82

Locked valid set: 498 turns/99 dialogues; Baseline 4503.14, Full 1155.51; reduction 74.34% [72.62%, 75.76%].
Seven externally contaminated executions from the 505-turn candidate pool were excluded after run-log review.

TABLE V
HISTORICAL-LATCHED-POLICY CONTEXT ACCURACY AND POOLED ASR TIME ON THE ORIGINAL PLATFORM; CELLS SHOW MEAN $\pm$ STD [P95/P99]

Configuration	MultiWOZ WER	MultiWOZ CER	CrossWOZ WER	CrossWOZ CER	Pooled ASR time (ms)
Prefix 1 + suffix 1	.0895 $\pm$.0614 [.1960/.2482]	.0196 $\pm$.0208 [.0611/.0799]	.0083 $\pm$.0160 [.0431/.0659]	.0203 $\pm$.0276 [.0747/.1054]	1327.48 $\pm$ 645.03 [2716.80/3049.16]
Prefix 1 + suffix 0	.0813 $\pm$.0611 [.1882/.2465]	.0221 $\pm$.0218 [.0603/.0842]	.0118 $\pm$.0188 [.0482/.0723]	.0292 $\pm$.0339 [.0927/.1193]	1224.96 $\pm$ 655.85 [2726.57/3018.06]
Prefix 0 + suffix 0	.0619 $\pm$.0678 [.1756/.3086]	.0226 $\pm$.0265 [.0657/.1072]	.0210 $\pm$.0237 [.0633/.0870]	.0500 $\pm$.0374 [.1146/.1316]	1086.16 $\pm$ 690.43 [2641.87/2973.93]

TABLE VI
HISTORICAL-POLICY POST-FEED RESIDUAL TTFT AND ERROR ON CONCATENATED HUMAN-READ SPEECH (SECOND PLATFORM)

Language	Condition (n)	A post-feed TTFT	B post-feed TTFT	A error	B error
English	Clean (75)	2805.9	1672.1	.0297 [.0290]	.0565 [.0499]
English	SNR 20 dB (30)	2228.8	1633.1	.0288 [.0270]	.0627 [.0573]
English	SNR 15 dB (30)	2334.3	1801.5	.0255 [.0264]	.0578 [.0556]
English	SNR 10 dB (30)	2201.4	1654.3	.0372 [.0387]	.0817 [.0799]
English	Speed 0.9 $\times$ (30)	2341.8	1602.4	.0458 [.0491]	.0751 [.0751]
English	Speed 1.1 $\times$ (30)	2181.7	1657.0	.0380 [.0440]	.0741 [.0767]
English	Babble (30)	2325.8	3425.1	.0271 [.0296]	.4372 [.4001]
Chinese	Clean (75)	2871.7	1698.8	.1077 [.1118]	.1180 [.1255]
Chinese	SNR 20 dB (30)	2294.6	1696.1	.1224 [.1281]	.1242 [.1299]
Chinese	SNR 15 dB (30)	2290.5	1630.6	.1064 [.1137]	.1110 [.1173]
Chinese	SNR 10 dB (30)	2311.8	1654.7	.1233 [.1275]	.1312 [.1405]
Chinese	Speed 0.9 $\times$ (30)	2236.5	1539.1	.1232 [.1306]	.1380 [.1392]
Chinese	Speed 1.1 $\times$ (30)	2125.4	1623.6	.1510 [.1612]	.1827 [.1918]
Chinese	Babble (30)	2227.7	2082.2	.1360 [.1429]	.2863 [.2662]

TTFT is mean ms. Error is mean-utterance [corpus] WER for English and CER for Chinese.

whereas the configured LA-2-style implementation decodes according to its own new-audio/agreement policy. This is one project-internal operating point rather than a claim about the LocalAgreement method family generally.

As shown in Table VII, our configured LA-2-style implementation at this operating point has a 34% larger mean post-feed residual TTFT than System B (2115.0 versus 1573.9 ms). The paired mean difference is 541.1 ms, with dialogue-cluster bootstrap 95% CI [485.66, 599.14] and a dialogue-level Wilcoxon $p = 6.24 \times 10^{-18}$. Recognition errors are descriptively similar in scale but differ across languages and systems; no quality equivalence is established. Repeated retained-buffer decoding also has higher latency on longer input at this configured point: 2199.7–2230.4 ms for Very Long and Extra Long samples versus 1551.1–1637.5 ms for System B. These results do not establish general superiority over the LocalAgreement family.

F. Commitment, Tokenizer Seams, and Semantics

Instrumentation on 50 fixed Very Long accumulated turns from 40 source dialogues records 375 downstream commits and zero rollback deliveries. Internal re-recognition nevertheless changes previously committed-region hypotheses 224

times, affecting 224/425 committed segments (52.7%) and 49/50 samples. Character edit distance has mean 2.3, P90 6, and maximum 16; normalized edit ratio has mean 5.6% and maximum 47.1%. The changes include lexical drift but remain invisible to downstream consumers. Concatenated System B text differs from System A full transcription by mean WER 4.93% and CER 2.69%.

Fragment-wise and one-shot token sequences differ at seams in 25/50 samples, with means of 4.36 differing blocks and 5.60 one-shot-side tokens among affected samples. Decoding both token sequences reconstructs identical text in all 50 cases. This supports text-level reconstruction only; it does not establish token, KV-state, or output-distribution equivalence.

Response quality is explored on the same 25 Chinese and 25 English synthetic samples. Replies are independently sampled with the 128-token cap. CLS embeddings from the locally downloaded BAAI/bge-m3 artifact [29], followed by L2 normalization and cosine similarity, yield mean 0.8832 ± 0.0755 (minimum 0.6232, P10 0.7951). The archived artifact was not a versioned Hugging Face snapshot, so its source revision is unavailable. A paired-equivalence judge gives 2.96/5 and 40.0% at or above 4; this measure confounds independent recommendation choices and truncation. A separate blinded

TABLE VII
POST-FEED RESIDUAL TTFT FOR OUR CONFIGURED LA-2-STYLE IMPLEMENTATION ON THE 498-TURN/99-DIALOGUE FILTERED SUBSET

System	Overall post-feed TTFT	P50	P95	Long	Very Long	Extra Long	zh CER	en WER
System A	5310.8±2817.9	4780.9	10462.1	1958.2	3905.9	7697.5	.0674	.1527
System B	1573.9±411.6	1484.4	2296.5	1464.3	1551.1	1637.5	.0732	.1702
LA-2-style	2115.0±693.3	1948.5	3463.2	1741.0	2199.7	2230.4	.0835	.1566

TTFT values are ms; error values are mean-utterance metrics. Overall WER: A .0951, B .1047, LA .1073.

intent-satisfaction judge gives System A 3.10/5 and System B 3.04/5; the paired B–A difference is -0.06 with bootstrap 95% CI $[-0.34, 0.22]$. The judge is the Command Code service identifier `deepseek/deepseek-v4-flash` [30], accessed in August 2026 with temperature 0 and deterministic A/B order randomization. No immutable backend build identifier was available. This single-judge, 50-turn analysis is exploratory. It includes no emission-time committed WER/CER, entity/number/negation error classification, task-success test, user study, or prespecified noninferiority margin; consequently, it does not establish quality preservation, tokenizer-seam harmlessness, or usability equivalence.

G. Server-Side Whole-System Time to First Playable PCM

Table VIII uses only the R7 unified run on the second platform. Each mode has 25 Chinese-input and 25 English-input accumulated turns (50 turns from 40 source dialogues) on one monotonic clock. “Input language” denotes the source stratum, not necessarily the generated TTS language. System B sends its first completed sentence to TTS, whereas System A sends its capped full response. This whole-system policy comparison jointly changes ASR scheduling, response-readiness rule, generated text, stopping outcome, TTS text length, and TTS workload; it is not an architecture-only, isolated-TTS, or matched-text comparison. The local ASR/LLM pipeline uses in-process queues, TTS runs on localhost, and no client playback timestamp is measured.

Under these unequal server-side response/TTS policies, median time to first playable PCM is 3113.7 ms for System B and 22269.9 ms for System A, an 86.0% policy-bundle difference. Much of the mean gap occurs after the first token: System B prepares a first sentence and sends mean 17.74-character text to TTS, whereas System A prepares a capped full response averaging 204.10 characters. Consequently, the 3578.3 versus 13561.1 ms TTS components do not constitute a matched-text TTS comparison, and no architecture-only attribution is supported. The six components close exactly for all 100 records. The 133.0-ms $t_{\text{feed_to_close_wait}}$ is the complete feed-end-to-input-close interval; independent audit attributes approximately 132.68 ms to waiting before explicit flush, 0.21 ms to explicit flush, and 0.12 ms after flush. It is therefore not a flush-computation cost. On a ten-sample three-round subset, server-side TTFA CV has mean 7.73% and maximum 20.70%. Language-stratum P95 values at $n = 25$ are descriptive, and no client-side or user-perceived onset was measured.

VI. CONCLUSION AND OUTLOOK

A. Summary

This paper measures a scheduling cascade that moves chunked ASR and LLM prompt work into the audio-input interval while preserving the Whisper and Qwen2 weights. All reported streaming results use the historical latched startup-duration policy. On the locked valid 498-turn/99-dialogue synthetic set, original-platform post-feed residual TTFT falls by 74.34% with dialogue-cluster 95% CI [72.62%, 75.76%]; the same set gives 70.36% [68.50%, 71.90%] on the second platform. Seven executions from the 505-turn candidate pool were excluded after run-log review confirmed contamination by concurrent external programs. On the original platform, mean System B post-feed residual TTFT remains near 1.1 s from Long through Extra Long, and concatenated human-read Extra Long reductions are 67.5% on LibriSpeech and 65.7% on AISHELL-1. Ablation associates most of the valid-set contrast with streaming ASR; incremental-prefill contrasts are -23.01 , 2.73 , and 40.82 ms and do not establish a stable large independent benefit under Qwen2-7B. Our configured LA-2-style implementation has 34% higher mean latency at one operating point, not a general method-family disadvantage. Under unequal whole-system policies, server-side first-playable-PCM medians are 3.11 s for System B’s first sentence and 22.27 s for System A’s capped full response; this is not an architecture-only comparison.

The implementation provides an append-only downstream text contract rather than immutable internal recognition. Internal hypotheses drift in the instrumented subset, tokenizer seams change token segmentation in half the samples, and the limited semantic experiment does not establish quality or task equivalence. These findings bound the claim: the measured historical scheduler reduces post-feed residual latency for the locked valid long-input set, while short-input overhead, babble-induced tails, recognition changes, response policy, and synthesis workload remain material trade-offs. No result is reported for a corrected current-queue-duration trigger.

B. Limitations

The main 1,132-turn benchmark remains TTS-synthesized, purposively long, duration-mix dependent, and clustered within 100 source dialogues rather than representing 1,132 independent natural interactions. The added LibriSpeech/AISHELL-1 data are concatenated human-read utterances, not spontaneous dialogue, and some concatenation seams are audible. System B is slower for Short and Medium synthetic input, while babble is a demonstrated

TABLE VIII
SERVER-SIDE WHOLE-SYSTEM POLICY COMPARISON: B FIRST SENTENCE VERSUS A CAPPED FULL RESPONSE (MS)

System	Input language	Mean $\pm$ std	P50	P90	P95	n
System B	Chinese	3303.3 $\pm$ 2681.3	2603.0	3130.4	8299.2	25
System B	English	7660.5 $\pm$ 2781.2	7577.0	10940.4	11857.2	25
System B	All	5481.9 $\pm$ 3486.1	3113.7	10506.6	11656.3	50
System A	Chinese	22616.8 $\pm$ 2197.2	22161.4	25459.9	26874.8	25
System A	English	22234.7 $\pm$ 2850.4	22439.6	25274.4	26487.5	25
System A	All	22425.7 $\pm$ 2526.1	22269.9	25588.8	26887.4	50
Component means $\pm$ std for all languages						
Component	System B		System A			
Speech end $\rightarrow$ feed end	0.1 $\pm$ 0.0		0.1 $\pm$ 0.0			
Feed end $\rightarrow$ input close	133.0 $\pm$ 98.7		0.0 $\pm$ 0.0			
Input close $\rightarrow$ first token	1383.1 $\pm$ 278.8		4182.9 $\pm$ 898.3			
First token $\rightarrow$ policy text ready	387.0 $\pm$ 429.2		4681.1 $\pm$ 583.9			
Text ready $\rightarrow$ TTS request	0.4 $\pm$ 0.1		0.6 $\pm$ 0.1			
TTS request $\rightarrow$ playable PCM	3578.3 $\pm$ 3023.6		13561.1 $\pm$ 2513.6			
Closed sum	5481.9		22425.7			
Policy facts: mean TTS characters A/B=204.10/17.74; exact greetings A/B=0/50 vs. 25/50; max-token stops A/B=41/50 vs. 43/50; 19/25 English-input B texts contain no Latin letters.						

failure boundary because VAD over-triggering and empty ASR outputs create backlog and long tails. The locked 498-turn ablation set excludes seven executions whose run logs showed contamination by concurrent external programs. Legacy timing begins at waveform feed completion, uses fixed arm order, and measures the historical latched startup gate; a corrected current-queue trigger has not been evaluated.

Experiments use a single-machine queue topology and no unrelated GPU jobs, but R7 ASR and TTS share GPU0 while the LLM uses GPU1. Distributed networks, client playback, shared-service contention, concurrency, throughput, energy, and multi-user capacity are not represented. The downstream interface never rolls back text, although internal re-recognition frequently changes committed-region hypotheses. Fragment-wise tokenization differs from one-shot tokenization, so KV-state equivalence is not established. Semantic evidence covers only 50 accumulated turns from 40 dialogues, independently sampled replies, a 128-token cap, and one service judge without repeated ratings; no quality noninferiority, task-success, user-experience, or production-reliability conclusion follows. The unified server-side TTFA comparison uses first-sentence System B output versus capped-full-response System A output, with markedly different TTS text lengths and output-language behavior. Finally, the prototype remains half-duplex, a limitation discussed in work on turn-taking systems [31]. Because the modules communicate through text, it also discards non-textual acoustic cues; privacy, cache lifecycle, routing fairness, accessibility, and secure multi-tenant deployment remain outside the evaluated scope.

C. Future Work

Future work should compare the historical latched startup gate with a corrected current-queue-duration trigger on the same samples and use counterbalanced arm order. A policy-matched 2×2 TTFA design should cross serial/pipeline scheduling with first-sentence/full-response TTS and add client playback timestamps. Further work should evaluate

spontaneous, accented, microphone, and overlapping speech; add VAD back-pressure and adaptive segmentation for babble; and test committed-text task outcomes with repeated human and multi-judge assessment. Rollback-aware cache repair or retokenization could reconcile later ASR corrections and tokenizer seams. Platform-aware routing should be evaluated rather than inferred from a fixed duration threshold. Distributed deployment, contention, throughput, resource/energy measurements, larger LLMs, privacy-preserving cache lifecycle, and full-duplex interruption handling would clarify how the measured scheduling behavior transfers beyond the laboratory prototype.

ACKNOWLEDGMENT

The authors thank the School of Computer, Electronics and Information, Guangxi University, for providing the research environment that supported this work. The authors gratefully acknowledge the open-source projects that made this work possible, including OpenAI Whisper, Silero VAD, and CosyVoice. The authors also thank the anonymous reviewers for their constructive comments.

REFERENCES

- [1] OpenAI, “GPT-4o system card,” *arXiv preprint arXiv:2410.21276*, 2024.
- [2] R. Prabhavalkar, T. Hori, T. N. Sainath, R. Schlüter, and S. Watanabe, “End-to-end speech recognition: a survey,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 325–351, 2024.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30. Red Hook, NY, USA: Curran Associates, 2017, pp. 5998–6008.
- [4] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and R. Pang, “Conformer: convolution-augmented Transformer for speech recognition,” in *Interspeech 2020*. ISCA, 2020, pp. 5036–5040.
- [5] D. Liu, G. Spanakis, and J. Niehues, “Low-latency sequence-to-sequence speech recognition and translation by partial hypothesis selection,” in *Interspeech 2020*. ISCA, 2020, pp. 3620–3624.

- [6] D. Macháček, R. Dabre, and O. Bojar, “Turning Whisper into real-time transcription system,” in *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, 2023, pp. 17–24.
- [7] H. Wang, G. Hu, G. Lin, W.-Q. Zhang, and J. Li, “Simul-Whisper: attention-guided streaming Whisper with truncation detection,” in *InterSpeech 2024*. ISCA, 2024, pp. 4483–4487.
- [8] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202. PMLR, 2023, pp. 28 492–28 518.
- [9] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, “A review on edge large language models: design, execution, and applications,” *ACM Computing Surveys*, vol. 57, no. 8, pp. 1–35, 2025, art. no. 209.
- [10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th ACM Symposium on Operating Systems Principles*. Koblenz, Germany: ACM, 2023, pp. 611–626.
- [11] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, “Prompt Cache: modular attention reuse for low-latency inference,” in *Proceedings of Machine Learning and Systems*, vol. 6, 2024, pp. 325–338. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/file/a66caai1703fe34705a4368c3014c1966-Paper-Conference.pdf
- [12] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems*, vol. 37. Curran Associates, Inc., 2024, pp. 62 557–62 583.
- [13] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35. Neural Information Processing Systems Foundation, Inc., 2022, pp. 16 344–16 359.
- [14] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *The Twelfth International Conference on Learning Representations*. Vienna, Austria: OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=NG7sS51zVF>
- [15] N. Sethiya and C. K. Maurya, “End-to-end speech-to-text translation: a survey,” *Computer Speech & Language*, vol. 90, p. 101751, 2025.
- [16] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, and E. Chen, “A survey on multimodal large language models,” *National Science Review*, vol. 11, no. 12, p. nwae403, 2024.
- [17] Z. Xie and C. Wu, “Mini-Omni: language models can hear, talk while thinking in streaming,” *arXiv preprint arXiv:2408.16725*, 2024.
- [18] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, “Moshi: a speech-text foundation model for real-time dialogue,” *arXiv preprint arXiv:2410.00037*, 2024.
- [19] Q. Fang, S. Guo, Y. Zhou, Z. Ma, S. Zhang, and Y. Feng, “LLaMA-Omni: seamless speech interaction with large language models,” in *The Thirteenth International Conference on Learning Representations*. Singapore: OpenReview.net, 2025. [Online]. Available: <https://openreview.net/forum?id=PYmrUQmMEw>
- [20] OpenAI, “Whisper large-v3-turbo model card and configuration,” Hugging Face model repository, 2024, public configuration revision 41f01f3fe87f28c78e2fbf8b568835947dd65ed9; accessed August 22, 2026. [Online]. Available: <https://huggingface.co/openai/whisper-large-v3-turbo/tree/41f01f3fe87f28c78e2fbf8b568835947dd65ed9>
- [21] Silero Team, “Silero VAD: pre-trained enterprise-grade voice activity detector,” GitHub repository, 2024, MIT-licensed software; accessed August 22, 2026. [Online]. Available: <https://github.com/snakers4/silero-vad>
- [22] P. Budzianowski, T.-H. Wen, B.-H. Tseng, I. Casanueva, S. Ultes, O. Ramadan, and M. Gašić, “MultiWOZ - a large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 5016–5026.
- [23] Q. Zhu, K. Huang, Z. Zhang, X. Zhu, and M. Huang, “CrossWOZ: a large-scale Chinese cross-domain task-oriented dialogue dataset,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 281–295, 2020.
- [24] Z. Du, Q. Chen, S. Zhang, K. Hu, H. Lu, Y. Yang, H. Hu, S. Zheng, Y. Gu, Z. Ma, Z. Gao, and Z. Yan, “CosyVoice: a scalable multilingual zero-shot text-to-speech synthesizer based on supervised semantic tokens,” *arXiv preprint arXiv:2407.05407*, 2024.
- [25] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “LibriSpeech: an ASR corpus based on public domain audio books,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2015, pp. 5206–5210.
- [26] H. Bu, J. Du, X. Na, B. Wu, and H. Zheng, “AISHELL-1: an open-source Mandarin speech corpus and a speech recognition baseline,” in *2017 20th Conference of the Oriental Chapter of the International Coordinating Committee on Speech Databases and Speech I/O Systems and Assessment (O-COCOSDA)*. IEEE, 2017, pp. 1–5.
- [27] D. Snyder, G. Chen, and D. Povey, “MUSAN: a music, speech, and noise corpus,” *arXiv preprint arXiv:1510.08484*, 2015.
- [28] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou *et al.*, “Qwen2 technical report,” *arXiv preprint arXiv:2407.10671*, 2024.
- [29] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-Embedding: multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” in *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024, pp. 2318–2335.
- [30] Command Code, “DeepSeek V4 Flash (latest): pricing, benchmarks, and speed,” Command Code online model catalog and service documentation, Jul. 2026, service model identifier: deepseek/deepseek-v4-flash; page release date July 31, 2026; accessed August 22, 2026. [Online]. Available: <https://commandcode.ai/models/deepseek-v4-flash>
- [31] G. Skantze, “Turn-taking in conversational systems and human-robot interaction: a review,” *Computer Speech & Language*, vol. 67, p. 101178, 2021.